

Экзаменационный билет №1	2
Экзаменационный билет №2	2
Экзаменационный билет №3	4
Экзаменационный билет №4	7
Экзаменационный билет №5	9
Экзаменационный билет №6	11
Экзаменационный билет №7	13
Экзаменационный билет №8	16
Экзаменационный билет №9	18
Экзаменационный билет №10	21
Экзаменационный билет №11	23
Экзаменационный билет №12	24
Экзаменационный билет №13	26
Экзаменационный билет №15	30
Экзаменационный билет №16	32
Экзаменационный билет №17	34
Экзаменационный билет №18	36
Экзаменационный билет №19	38
Экзаменационный билет №20	39
Экзаменационный билет №21	42
Экзаменационный билет №22	43
Экзаменационный билет №23	45
Экзаменационный билет №24	47
Экзаменационный билет №25	48
Экзаменационный билет №26	51
Экзаменационный билет №27	53
Экзаменационный билет №28	54
Экзаменационный билет №29	55
Экзаменационный билет №30	58
Экзаменационный билет №31	61
Экзаменационный билет №32	63
Экзаменационный билет №33	65

Экзаменационный билет №1

1. Написать рекурсивную функцию: получение последовательности чисел Фибоначчи, если количество элементов в последовательности равно N. Составить блок-схему алгоритма и вписать код на C++.

```
#include <iostream>

int fibonacci(int n) {
    if (n <= 1) {
        return n;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int N;
    std::cin >> N;

    for (int i = 0; i < N; i++) {
        std::cout << fibonacci(i) << " ";
    }

    return 0;
}
```

2. Текстовые файлы: написать на C++ функцию, копирующую в файл F2 только те строки из F1, которые начинаются и заканчиваются на одну и ту же букву.

```
#include <iostream>
#include <string>
#include <fstream>

int main() {
    std::ifstream f1("F1.txt"); // Для чтения
    std::ofstream f2("F2.txt"); // Для записи

    std::string line = "";
    while (std::getline(f1, line)) {
        if (!line.empty() && line[0] == line[line.length() - 1]) {
            f2 << line << std::endl;
        }
    }

    f1.close();
    f2.close();
    return 0;
}
```

Экзаменационный билет №2

1. Написать рекурсивную функцию: считывать введенные с клавиатуры числа до тех пор, пока не будет введен ноль. Составить блок-схему алгоритма и вписать код на C++.

```

void func() {
    int n;
    std::cin >> n; // Считываем число с клавиатуры

    if (n != 0) {
        func(); // Рекурсивный вызов функции, если число не равно нулю
    }
}

```

2. Очередь в C++: написать блок кода для очереди, который будет имитировать работу касс в магазине. В магазине есть три кассы, которые работают параллельно. Каждые 5 секунд на каждую кассу поступает новый покупатель. Если на кассе есть свободный кассир, то покупатель обслуживается, иначе он становится в очередь.

```

// Очередь покупателей
std::queue<int> customersQueue;

// Функция для работы кассира
void cashier(int cashierNumber) {
    while (true) {
        // Кассир ждет 5 секунд
        std::this_thread::sleep_for(std::chrono::seconds(5));

        // Проверка, есть ли покупатели в очереди
        if (!customersQueue.empty()) {
            int customer = customersQueue.front(); // Получение номера покупателя
            customersQueue.pop(); // Удаление покупателя из очереди
            // Обслуживание покупателя
            std::cout << "Касса " << cashierNumber << ": Обслуживается покупатель " << customer << std::endl;
        } else {
            // Если очередь пуста, кассир сообщает об этом
            std::cout << "Касса " << cashierNumber << ": Нет покупателей в очереди" << std::endl;
        }
    }
}

```

Экзаменационный билет №3

1. Стек: понятие стека, отличие от списка, добавление и удаление элементов в стек. Для чего используется стек. Написать функцию на C++, вставляющую новый элемент на заданную позицию. Прокомментировать код.

Стек - это абстрактная структура данных, которая работает по принципу «последним пришел - первым вышел».

Работая со стеком, можно добавлять элементы на вершину стека и удалять их только с вершины.

Отличие стека от списка заключается в том, что *в стеке доступ осуществляется только к вершине стека*, в то время как *в списке можно получить доступ к элементам в любом месте*.

Стек используется для управления вызовами функций, в рекурсиях, управление памятью.

```
// Структура для узла стека
struct Node {
    int data; // Данные узла стека
    Node* next; // Указатель на следующий узел
};

Node* top = nullptr; // Указатель на вершину (и одновременно дно) стека (создаем стек)
int size = 0; // Размер стека

// Функция для вставки нового элемента на заданную позицию в стеке
void insert(int value, int position) {
    Node* temp = new Node; // Создаем новый узел
    temp->data = value; // Устанавливаем значение в новый узел

    if (position == 0) { // Вставка в начало стека
        temp->next = top; // Связываем новый узел со стеком
        top = temp; // Обновляем вершину стека
    }
    else {
        Node* current = top; // Определяем текущий узел как вершину стека
        // Поиск узла перед позицией вставки
        for (int i = 1; i < position; i++) {
            current = current->next; // Перемещаемся к узлу перед позицией вставки
        }
        temp->next = current->next; // Связываем новый узел с последующим узлом
        current->next = temp; // Обновляем порядок узлов в стеке
    }

    size++; // Увеличиваем размер стека после вставки
}
```

2. Составьте блок-схему с вписанным кодом на C++: метод двухфазной сортировки (сбалансированным слиянием). Ввести динамический массив целых чисел, количество элементов массива равно N. Отсортировать элементы массива методом «Двухфазной сортировки». Массив содержит следующие элементы: 9 7 6 15 16 5 10 11. Показать на заданных элементах последовательность шагов сортировки. Написать фрагмент кода сортировки на C++.

Алгоритм:



// Функция, которая объединяет два отсортированных подмассива в один отсортированный массив

```
void merge(std::vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1; // Размер первого подмассива
    int n2 = right - mid; // Размер второго подмассива

    std::vector<int> leftArray(n1), rightArray(n2);

    // Заполнение временных массивов
    for (int i = 0; i < n1; i++) {
        leftArray[i] = arr[left + i];
    }
    for (int j = 0; j < n2; j++) {
        rightArray[j] = arr[mid + 1 + j];
    }

    // Объединение двух массивов в один отсортированный массив
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (leftArray[i] <= rightArray[j]) {
            arr[k] = leftArray[i];
            i++;
        }
        else {
            arr[k] = rightArray[j];
            j++;
        }
        k++;
    }
}
```

```

        // Завершение слияния, если остались элементы в левом и правом массивах
        while (i < n1) {
            arr[k] = leftArray[i];
            i++;
            k++;
        }

        while (j < n2) {
            arr[k] = rightArray[j];
            j++;
            k++;
        }
    }

    // Функция для двухфазной сортировки слиянием
    void mergeSort(std::vector<int>& arr, int left, int right) {
        if (left < right) {
            int mid = left + (right - left) / 2;

            mergeSort(arr, left, mid); // Сортировка левой половины массива
            mergeSort(arr, mid + 1, right); // Сортировка правой половины массива

            merge(arr, left, mid, right); // Слияние двух половин
        }
    }
}

```

Экзаменационный билет №4

1. Функции в C++: написать функцию, которая принимает в качестве параметра и возвращает строку, заменяющую все «1» на «0». Вызвать эту функцию для строки, введённой с клавиатуры. Привести пример кода с комментариями.

```
#include <iostream>
#include <string>

std::string func(std::string& input) {
    for (int i = 0; i < input.length(); i++) {
        if (input[i] == '1') {
            input[i] = '0';
        }
    }

    return input;
}

int main() {
    std::string s;
    std::getline(std::cin, s);

    std::string result = func(s);

    std::cout << result << std::endl;

    return 0;
}
```

2. Как получить доступ к списку параметров через указатель в функции с переменным числом параметров? Приведите пример кода, демонстрирующего работу с указателем на список параметров.

Для работы с переменным числом параметров обычно используется механизм "variadic templates" (шаблоны с переменным числом параметров) или "variadic functions" (функции с переменным числом параметров).

```
#include<iostream>
#include<stdarg.h> // Включаем библиотеку для работы с переменным количеством
аргументов
using namespace std;

// Функция с переменным количеством аргументов
int func(int count, ...) {
    va_list arg; // Создаем переменную для работы с аргументами
    va_start(arg, count); // Инициализируем список аргументов
    int n = 0; // Переменная для суммирования значений
    for (int i = 0; i < count; ++i)
        n += va_arg(arg, int); // Получаем конкретный аргумент из списка и
прибавляем к сумме
    va_end(arg); // Завершаем работу с аргументами
```

```
        return n; // Возвращаем сумму
    }

int main()
{
    cout << func(3, 5, 6, 7); // Вызов функции с тремя аргументами (сумма: 5
+ 6 + 7)
}
```


Экзаменационный билет №5

1. Составьте блок-схему с вписанным кодом на C++: метод двухфазной сортировки (сбалансированным слиянием). Ввести динамический массив целых чисел, количество элементов массива равно N. Отсортировать элементы массива методом «Двухфазной сортировки». Массив содержит следующие элементы: 9 7 6 15 16 5 10 11. Показать на заданных элементах последовательность шагов сортировки. Написать фрагмент кода сортировки на C++.



```
// Функция, которая объединяет два отсортированных подмассива в один
отсортированный массив
void merge(std::vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1; // Размер первого подмассива
    int n2 = right - mid; // Размер второго подмассива

    std::vector<int> leftArray(n1), rightArray(n2);

    // Заполнение временных массивов
    for (int i = 0; i < n1; i++) {
        leftArray[i] = arr[left + i];
    }
    for (int j = 0; j < n2; j++) {
        rightArray[j] = arr[mid + 1 + j];
    }

    // Объединение двух массивов в один отсортированный массив
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (leftArray[i] <= rightArray[j]) {
            arr[k] = leftArray[i];
            i++;
        }
        else {
            arr[k] = rightArray[j];
            j++;
        }
        k++;
    }
}
```

```

        // Завершение слияния, если остались элементы в левом и правом массивах
        while (i < n1) {
            arr[k] = leftArray[i];
            i++;
            k++;
        }

        while (j < n2) {
            arr[k] = rightArray[j];
            j++;
            k++;
        }
    }

    // Функция для двухфазной сортировки слиянием
    void mergeSort(std::vector<int>& arr, int left, int right) {
        if (left < right) {
            int mid = left + (right - left) / 2;

            mergeSort(arr, left, mid); // Сортировка левой половины массива
            mergeSort(arr, mid + 1, right); // Сортировка правой половины массива

            merge(arr, left, mid, right); // Слияние двух половин
        }
    }
}

```

2. Написать рекурсивную функцию: считывать введенные с клавиатуры числа до тех пор, пока не будет введен ноль. Составить блок-схему алгоритма и вписать код на C++.

```

void func() {
    int n;
    std::cin >> n; // Считываем число с клавиатуры

    if (n != 0) {
        func(); // Рекурсивный вызов функции, если число не равно нулю
    }
}

```

Экзаменационный билет №6

1. Динамические массивы: Объявление, какой областью памяти является, дать определение динамического массива, как обращаться к элементам динамического массива. Ввести динамический массив целых чисел, количество элементов массива равно N. Перевернуть элементы массива, начиная с элемента с индексом P, заканчивая элементом с индексом Q. Составить блок-схему алгоритма и вписать код на C++.

Динамический массив – структура данных, которая может изменять свой размер во время выполнения программы и освобождать ненужные ячейки памяти.

Объявляется с использованием указателя на первый элемент массива: `int* arr = new int[size];`

Оператор `new[]` помогает выделить память в динамической памяти: запрашивает соответствующее количество байтов у ОС и возвращает указатель в начало области.

`[]` - интерпретирует область памяти как последовательность элементов массива.

`delete[]` - используется, чтобы не произошла утечка памяти.

Обращение к элементу 0: `arr[0]`, `*arr`

Обращение к элементу 1: `*(arr + 1)`

```
#include <iostream>
```

```
// Функция для переворачивания элементов массива от P до Q
```

```
void reverse(int* array, int start, int end) {
```

```
    while (start < end) {
```

```
        int temp = array[start];
```

```
        array[start] = array[end];
```

```
        array[end] = temp;
```

```
        start++;
```

```
        end--;
```

```
    }
```

```
}
```

```
int main() {
```

```
    int N = 5; // Количество элементов
```

```
    int* mas = new int[N]; // Выделение памяти под динамический массив
```

```
    // Инициализация массива (для примера)
```

```
    for (int i = 0; i < N; i++) {
```

```
        mas[i] = i + 1;
```

```
    }
```

```
    int P = 1, Q = 3; // Пример индексов P и Q
```

```
    reverse(mas, P, Q); // Переворачивание элементов от P до Q (включительно)
```

```
    // Вывод элементов массива для проверки
```

```
    for (int i = 0; i < N; i++) {
```

```
        std::cout << mas[i] << " ";
```

```
    }
```

```
    delete[] mas; // Освобождение памяти массива
```

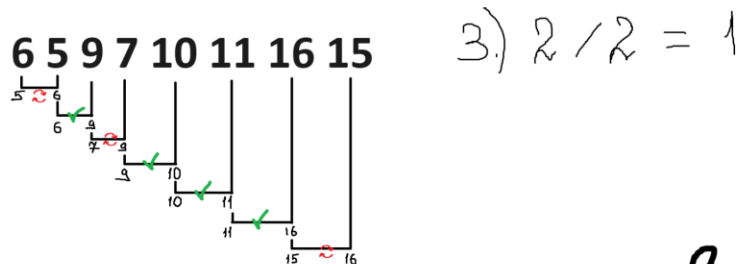
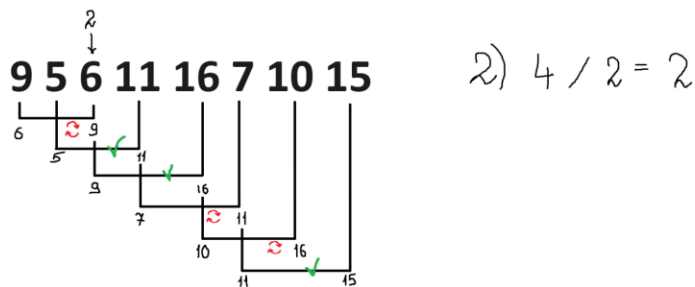
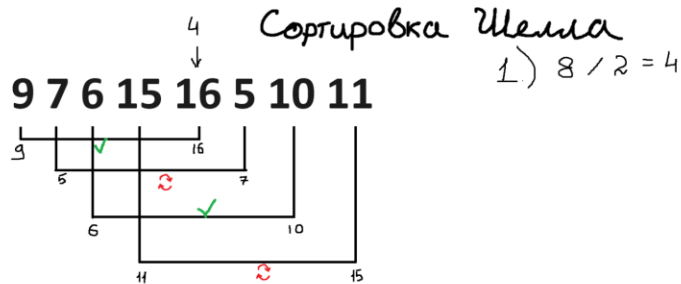
```
    return 0;  
}
```

2. Написать рекурсивную функцию: «Ханойская башня» для трёх стержней на C++. Составить блок-схему алгоритма и вписать код на C++.

```
#include <iostream>  
  
void towerOfHanoi(int n, int start, int finish, int temp) {  
    if (n == 1) {  
        std::cout << start << " to " << finish << std::endl;  
        return;  
    }  
    towerOfHanoi(n - 1, start, temp, finish);  
    std::cout << start << " to " << finish << std::endl;  
    towerOfHanoi(n - 1, temp, finish, start);  
}  
  
int main() {  
    towerOfHanoi(3, 1, 3, 2); // Вызов функции Ханойской башни  
  
    return 0;  
}
```

Экзаменационный билет №7

1. Составьте блок-схему с вписанным кодом на C++: метод сортировки «Шелла». Ввести динамический массив целых чисел, количество элементов массива равно N. Отсортировать элементы массива методом «Шелла». Массив содержит следующие элементы: 9 7 6 15 16 5 10 11. Показать на заданных элементах последовательность шагов сортировки. Написать фрагмент кода сортировки на C++.



5 6 7 9 10 11 15 16

✓ готово

```
void ShellSort(int arr[], int lenght) {  
    for (int step = lenght / 2; step > 0; step /= 2) {  
        for (int i = step; i < lenght; ++i) {  
            for (int j = i - step; j >= 0 && arr[j] > arr[j + step]; j -= step) {  
                swap(arr[j], arr[j + step]);  
            }  
        }  
    }  
}
```

2. Функция `getline`. Показать её использование в C++ на примере задания: вывести все слова из строки, которые не содержат букву А.

```

#include <iostream>
#include <string>

int main() {
    std::string s;
    std::string word = "";
    std::getline(std::cin, s);

    bool flag = false;
    s = s + " "; // Добавляем пробел в конце строки, чтобы обработать
последнее слово
    for (int i = 0; i < s.length(); i++) {
        if (s[i] != ' ') {
            word += s[i];
        }
        else {
            flag = false;
            for (int j = 0; j < word.length(); j++) {
                if (word[j] == 'A' || word[j] == 'a') {
                    flag = true;
                    break;
                }
            }
            if (!flag) {
                std::cout << word << std::endl;
            }
            word = "";
        }
    }

    return 0;
}

```

```

#include <iostream>
#include <string>

int main() {
    std::string s;
    std::string word = "";
    std::getline(std::cin, s);

    bool flag = false;
    s = s + ' '; // Добавляем пробел в конце строки, чтобы обработать последнее
слово

    for (char c : s) {
        if (c != ' ') {
            word += c;
        } else {

```

```
        flag = false;
        for (char letter : word) {
            if (letter == 'A' || letter == 'a') {
                flag = true;
                break;
            }
        }
        if (!flag) {
            std::cout << word << std::endl;
        }
        word = "";
    }
}

return 0;
}
```

Экзаменационный билет №8

1. Функции в C++: написать функцию, которая принимает в качестве параметра и возвращает двумерный массив целых чисел, удаляющую все строки, в которых все значения равны нулю. Вызвать эту функцию для массива, введенного с клавиатуры. Привести пример кода с комментариями.

```
#include <iostream>
#include <vector>

const int n = 3;

int main()
{
    // ↓↓↓ Ввод двумерного массива ↓↓↓
    std::vector<std::vector<int>> vec(n, std::vector<int>(n));
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            std::cin >> vec[i][j];
        }
    }
    // ↑↑↑ Ввод двумерного массива ↑↑↑
    std::cout << std::endl;
    for (int i = 0; i < vec.size(); i++) // ←←← Проходим весь массив по строкам
    {
        bool flag = true; // Флаг (Все нули 000)
        for (int j = 0; j < n; j++) // ←←← Проходим весь массив по столбцам
        {
            if (vec[i][j] != 0) flag = false;
        }
        if (flag == true) {
            vec.erase(vec.begin() + i); // Удаление строки, в которой все нули
            --i; // Надо :)
        }
    }
    for (int i = 0; i < vec.size(); i++) // ←←← Выводим новый массив
    {
        for (int j = 0; j < vec[i].size(); j++)
        {
            std::cout << vec[i][j] << ' ';
        }
        std::cout << std::endl;
    }
    return 0;
}
```

2. Очередь: понятие очереди, отличие от списка, добавление и удаление элементов в очередь. Для чего используется очередь. Написать функцию на C++, вставляющую новый элемент на заданную позицию. Прокомментировать код.

Контейнер, который работает по принципу первый вошел — первым вышел. Первым всегда извлекается первый добавленный элемент.

Отличие от списка: добавление элементов – в конец, удаление элементов – из начала.

Очереди используются в обслуживании клиентов, в сети (передача кадров данных), в многозадачности.

Код:

```
int push(int newData, int position){
    node* newNode = new node;
    newNode->data = newData;

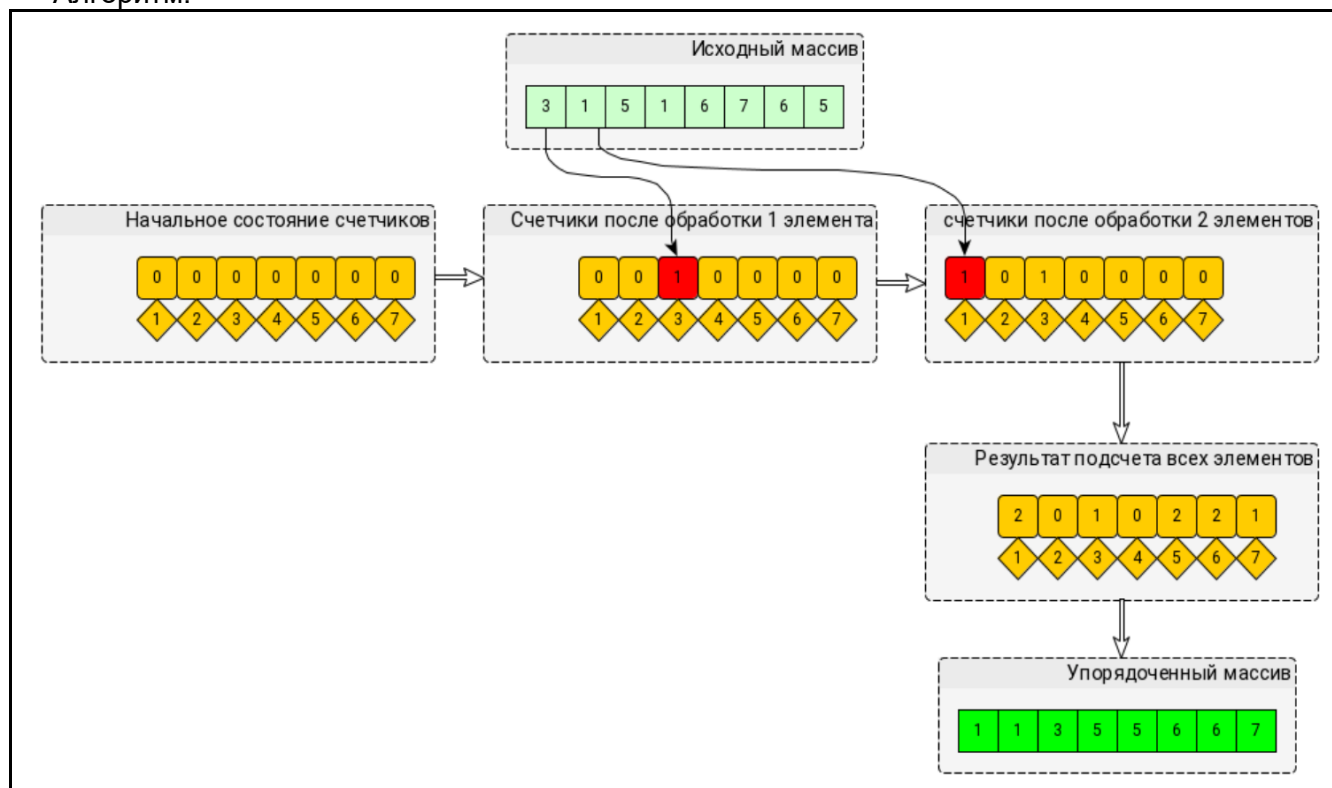
    if (position == 0){ // Если нужно вставить на первую позицию
        newNode->next = front;
        front = newNode;
        if (front == nullptr){
            back = newNode;
        }
        return;
    }

    node* current = front;
    int currentPos = 0;
    while (current != nullptr && currentPos < position - 1){ // Перебор всей
очереди до нужного эл-та
        current = current->next;
        currentPos++;
    }
    newNode->next = current->next;
    current->next = newNode;
    if (current == back){
        back = newNode;
    }
}
```

Экзаменационный билет №9

1. Составьте блок-схему с вписанным кодом на C++: метод сортировки подсчетом. Ввести динамический массив целых чисел, количество элементов массива равно N. Отсортировать элементы массива методом «Подсчета». Массив содержит следующие элементы: 9 7 6 15 16 5 10 11. Показать на заданных элементах последовательность шагов сортировки.

Алгоритм:



На заданном массиве:

1)

8	7	6	5	16	5	10	11
---	---	---	---	----	---	----	----

 $\max = 16$

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

2)

0	0	0	0	1	1	1	0	1	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

5	6	7	9	10	11	15	16
---	---	---	---	----	----	----	----

Код:

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> A = { 3, 3, 3, 1, 1, 0, 2, 2 };

    int max = A[0]; // Находим максимальный элемент
    for (int i = 1; i < A.size(); i++) {
```

```

        if (A[i] > max) {
            max = A[i];
        }
    }

    // Создаем вектор, размер которого равен максимальному элементу + 1, т.к у
нас индексы
    vector<int> B(max + 1, 0);
    for (int i = 0; i < A.size(); i++) { // Идем по исходному массиву. Если
встретили 0, то к нулевому индексу массива B += 1, если встретили 1, то к 1-ому
индексу массива B += 1
        int index = A[i];
        B[index] += 1;
    }

    // Переформировываем массив
    int k = 0;
    for (int i = 0; i < max + 1; i++) {
        for (int j = 0; j < B[i]; j++) {
            A[k] = i;
            k++;
        }
    }

    // Выводим
    for (int i = 0; i < A.size(); i++) {
        cout << A[i] << " ";
    }

    return 0;
}

```

2. Использование указателей в динамическом массиве. Сформировать динамический массив, инициализировать его, добавить после каждого элемента с чётным значением элемент, равный Z. Продемонстрировать операцию разыменования и разадресации.

```

#include <iostream>
#include <cstdlib>

int main() {
    int n = 5;
    int* mas = new int[n];

    for (int i = 0; i < n; ++i) {
        mas[i] = rand() % 10;
    }
}

```

```

    int newSize = n; // Используем отдельную переменную для хранения нового
размера массива

    for (int i = 0; i < n; ++i) {
        if (mas[i] % 2 == 0) {
            newSize++; // Увеличиваем размер массива на 1
        }
    }

    int* newMas = new int[newSize]; // Создаем новый массив с увеличенным
размером
    int index = 0;

    for (int i = 0; i < n; ++i) {
        newMas[index] = mas[i];
        if (mas[i] % 2 == 0) {
            index++;
            newMas[index] = -1; // Вставляем после четного числа
        }
        index++;
    }

    delete[] mas; // Освобождаем память, выделенную под исходный массив
    mas = newMas; // Переносим указатель на новый массив

    // Выводим результат
    for (int i = 0; i < newSize; ++i) {
        std::cout << mas[i] << " ";
    }

    std::cout << std::endl;

    // Продемонстрируем операцию разыменования и разадресации указателей
    int* pointer = mas; // Указатель на начало массива
    std::cout << pointer << std::endl;

    pointer++; // Увеличиваем указатель на 1 -> смещаемся на одну позицию в
массиве
    std::cout << *pointer << std::endl;

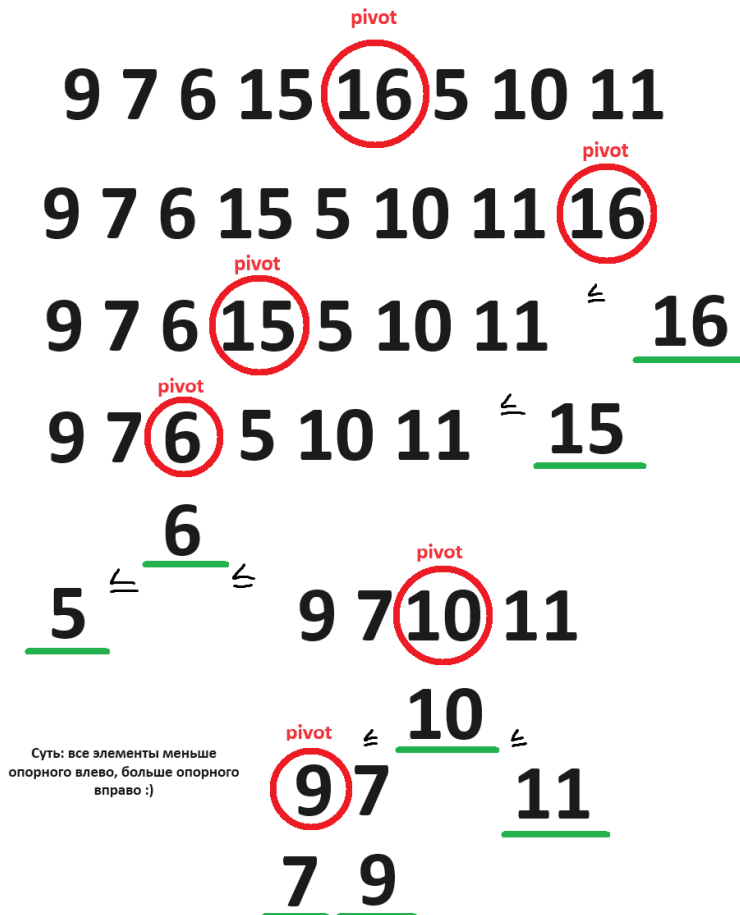
    delete[] mas; // Освобождаем память, выделенную под новый массив

    return 0;
}

```

Экзаменационный билет №10

1. Составьте блок-схему с вписанным кодом на C++: метод сортировки Хоара. Ввести динамический массив целых чисел, количество элементов массива равно N. Отсортировать элементы массива методом «Хоара». Массив содержит следующие элементы: 9 7 6 15 16 5 10 11. Показать на заданных элементах последовательность шагов сортировки.



5 6 7 9 10 11 15 16

```
int partOfSortHoara(int start, int end, vector<int>& vect) {
    int mid_element = vect[(start + end) / 2];
    int st = start, en = end;
    while (st <= en) {
        while (vect[st] < mid_element)
            st++;
        while (vect[en] > mid_element)
            en--;
        if (st <= en) {
```

```

        swap(vect[st], vect[en]);
        st++;
        en--;
    }
}
return st;
}
void sortHoara(int start, int end, vector<int>& vect) {
    if (end - start < 1) return;
    int border = partOfSortHoara(start, end, vect);
    sortHoara(start, border - 1, vect);
    sortHoara(border, end, vect);
}

```

- Поиск данных. ХЭШ-таблицы, свойства. ХЭШ-функция. Привести определения терминов и примеры кода на C++.

Поиск – процесс нахождения конкретной информации в ранее созданном множестве данных.
Хеш-таблица – абстрактная структура данных с функционалом массивов и списков.

- Неупорядоченная структура
- Состоит из хеш-функции (хеш-кода) и массива для хранения
- Хеш-функция возвращает число, позволяющее определить место для размещения элемента.

Хеш-функция – алгоритм для преобразования строки в число (в пределах массива). Чем меньше таблица и больше данных, тем больше коллизий (разные данные с одним хеш-кодом).

```

unsigned int hash(char* str)
{
    int sum = 0;
    for (int j = 0; str[j] != '\0'; j++)
    {
        sum += str[j];
    }
    return sum % HASH_MAX;
}

```

Экзаменационный билет №11

1. Интерполяционный метод поиска: В массиве найти заданный элемент равный z методом интерполяционного поиска. Реализовать фрагмент кода на C++ интерполяционным методом поиска: Дан отсортированный массив целых чисел и искомое число. Определить, есть ли искомое число в массиве. Пример: массив [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], искомое число 7. Ответ: есть.

```
bool InterpolationSearch(vector<int>& arr, int num) {
    int low = 0, high = arr.size()-1;
    int index;
    while (arr[low] < num && arr[high] > num) {
        if (arr[low] == arr[high])
            break;
        index = (num - arr[low]) * (low - high) / (arr[low] - arr[high]) +
low;
        if (arr[index] > num)
            high = index - 1;
        if (arr[index] < num)
            low = index + 1;
        else
            return true;
    }
    return false;
}
```

2. Рекурсия: рекурсивные функции, виды рекурсии. Привести определения терминов и примеры кода на C++.

Рекурсивная функция - это функция, которая вызывает саму себя внутри своего тела.

Виды рекурсии:

1. Простая рекурсия. Когда функция вызывает саму себя без вложенных вызовов других функций.
2. Косвенная рекурсия. Когда функции вызывают друг друга циклически. Пример: функция А вызывает функцию В, которая в свою очередь вызывает функцию А и т.д.
3. Множественная рекурсия. Когда функция вызывает саму себя несколько раз внутри своего тела. Пример: создание дерева.
4. Хвостовая рекурсия. Когда функция вызывает саму себя в качестве последней операции внутри своего тела. Это позволяет оптимизировать код и избежать переполнение.

Экзаменационный билет №12

1. Динамические массивы: Объявление, какой областью памяти является, дать определение динамического массива, как обращаться к элементам динамического массива. Ввести динамический массив целых чисел, количество элементов массива равно N. Проверить, упорядочены ли элементы массива по возрастанию. Составить блок-схему алгоритма и вписать код на C++.

Объявление динамического массива:

```
int* dynamicArray = new int[N];
```

Областью памяти, занимаемой динамическим массивом, является куча (heap). Когда вы создаете динамический массив, операционная система выделяет блок памяти из кучи для хранения элементов массива.

Обращение к элементам динамического массива осуществляется так же, как и к элементам обычного массива, используя квадратные скобки:

```
dynamicArray[i] = 5; // Присваивание значения элементу массива
```

```
int x = dynamicArray[j]; // Получение значения элемента массива
```

```
#include <iostream>
```

```
int main() {
    int N;
    std::cin >> N;

    int* dynamicArray = new int[N];

    for (int i = 0; i < N; ++i) {
        std::cin >> dynamicArray[i];
    }

    bool flag = true;
    for (int i = 0; i < N - 1; ++i) {
        if (dynamicArray[i] > dynamicArray[i + 1]) {
            flag = false;
            break;
        }
    }

    if (flag) {
        std::cout << "Yes" << std::endl;
    }
    else {
        std::cout << "No" << std::endl;
    }

    delete[] dynamicArray;
```



```
    return 0;
}
```

2. Функции в C++: написать функцию, которая принимает в качестве параметра и возвращает одномерный массив строк, удаляющую последний элемент в строках, имеющих нечётную длину. Вызвать эту функцию для массива, введённого с клавиатуры. Привести пример кода с комментариями.

```
void delLast (string& arr) {
    string word = "";
    if (arr.length() % 2 == 1) {
        for (int i = 0; i < arr.length()-1; i++) {
            word += arr[i];
        }
        arr = word;
    }
}
```

Экзаменационный билет №13

1. Строковые и символьные переменные в C++. Передача строк в функцию как фактических параметров. Привести определения терминов и пример кода на C++.

Передача строк в функцию как фактических параметров:

Строки могут быть переданы в функцию как фактические параметры двумя способами: по значению и по ссылке.

При передаче строки *по значению* создается копия строки, которая затем передается в функцию. Это может занимать много времени и памяти для больших строк.

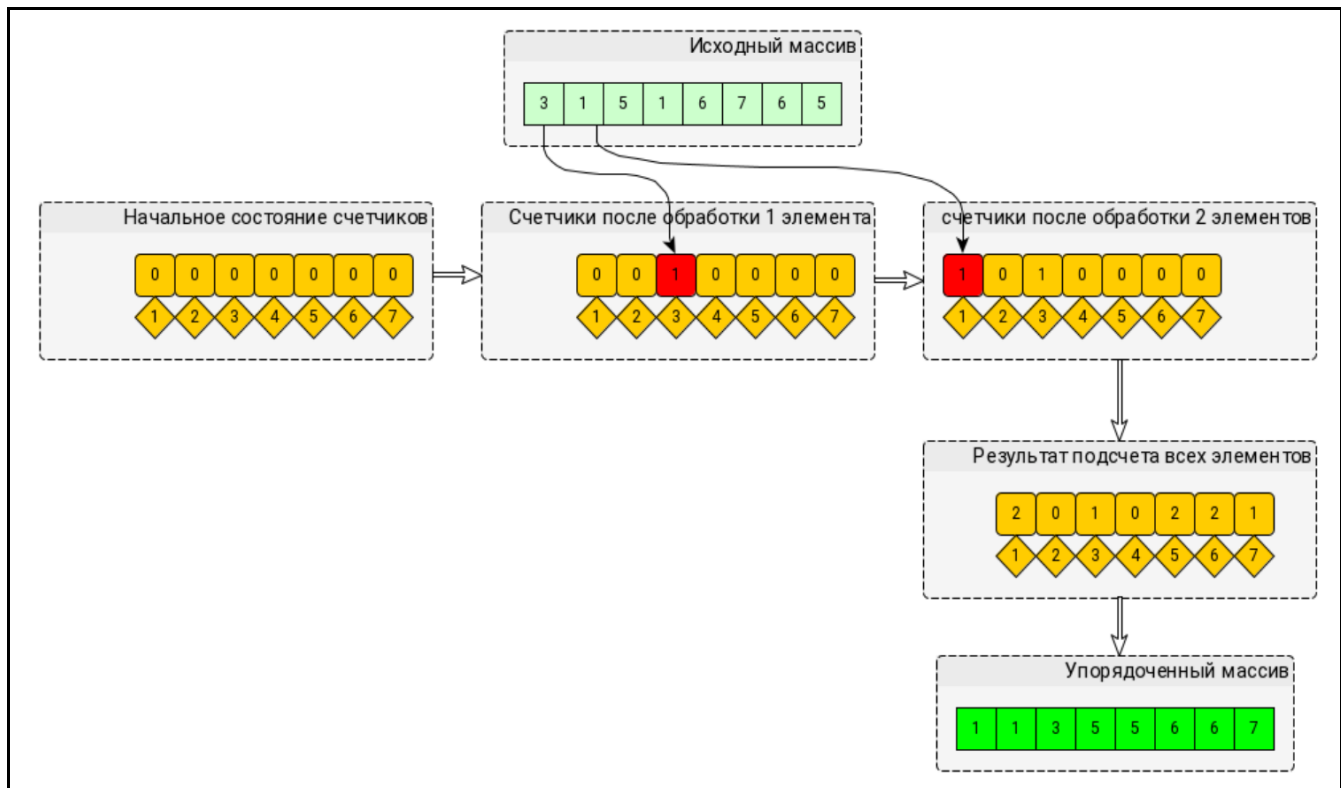
```
void print(const string str) {...}
```

При передаче строки *по ссылке* передается адрес строки, а не копия, что позволяет избежать затрат на создание копий и работать с оригинальной строкой.

```
void print(const string& str) {...}
```

2. Составьте блок-схему с вписанным кодом на C++: метод сортировки подсчетом. Ввести динамический массив целых чисел, количество элементов массива равно N. Отсортировать элементы массива методом «Подсчета». Массив содержит следующие элементы: 9 7 6 15 16 5 10 11. Показать на заданных элементах последовательность шагов сортировки.

Алгоритм:



На заданном массиве:

1)	8	7	6	5	16	5	10	11		max = 16
	0	0	0	0	0	0	0	0	0	0
2)	0	0	0	0	1	0	1	0	1	1
	5	6	7	8	10	11	15	16		

Код:

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> A = { 3, 3, 3, 1, 1, 0, 2, 2 };

    int max = A[0]; // Находим максимальный элемент
    for (int i = 1; i < A.size(); i++) {
        if (A[i] > max) {
            max = A[i];
        }
    }

    // Создаем вектор, размер которого равен максимальному элементу + 1, т.к у
    нас индексы
    vector<int> B(max + 1, 0);
    for (int i = 0; i < A.size(); i++) { // Идем по исходному массиву. Если
    встретили 0, то к нулевому индексу массива B += 1, если встретили 1, то к 1-ому
    индексу массива B += 1
        int index = A[i];
        B[index] += 1;
    }

    // Переформировываем массив
    int k = 0;
    for (int i = 0; i < max + 1; i++) {
        for (int j = 0; j < B[i]; j++) {
            A[k] = i;
            k++;
        }
    }

    // Выводим
    for (int i = 0; i < A.size(); i++) {
        cout << A[i] << " ";
    }

    return 0;
}
```

Экзаменационный билет №14

1. Список: понятие списка, отличие однонаправленных и двунаправленных списков, добавление и удаление элементов в список. Для чего используется список. Написать функцию на C++, вставляющую новый элемент на заданную позицию. Прокомментировать код.

Список – динамическая структура данных, позволяющая хранить данные. При этом не обладает недостатками в организации памяти.

Однонаправленный список – содержит данные и указывает только на следующий элемент списка (от начала в конец).

Двунаправленный список – содержит данные и указывает на следующий и предыдущий элемент списка (движение в обе стороны).

При добавлении элемент размещается где угодно в памяти.

В каждом элементе хранится адрес следующего элемента. Нужно просто переобозначить указатели на новые элементы после вставки.

При удалении ситуация аналогичная: изменяем указатели в предыдущем элементе и очищаем ячейку.

```
void insertElement(Node*& head, int element, int position) {
    Node* newElement = new Node(element);
    Node* temp = head;
    for (int i = 0; i < position - 1 && temp != nullptr; i++) {
        temp = temp->next;
    }
    newElement->next = temp->next;
    temp->next = newElement;
}
```

2. Структуры в C++. Создать структуру “Государство”, которая содержит 4 поля: название, столица, численность населения, занимаемая площадь. Привести блок кода создания структуры и написать функцию, удаляющую из списка таких структур те, численность населения в которых меньше заданной.

```
#include <iostream>
#include <list>
#include <string>

struct Country {
    std::string name;
    std::string capital;
    int population;
    int area;
};

int main()
```

```

{
    Country country1 = { "Russia", "Moscow", 222, 2323 };
    Country country2 = { "USA", "Washington", 333, 1111 };
    Country country3 = { "Canada", "Ottawa", 99, 2222 };
    std::list<Country> lst;
    lst.push_back(country1);
    lst.push_back(country2);
    lst.push_back(country3);
    for (Country tmp : lst)
    {
        if (tmp.population > 100)
        {
            std::cout << tmp.name << " naselenie " << tmp.population <<
std::endl;
        }
        else
        {
            lst.erase(lst.begin(), lst.begin());
        }
    }
    return 0;
}

```

Экзаменационный билет №15

1. Динамические массивы: Объявление, какой областью памяти является, дать определение динамического массива, как обращаться к элементам динамического массива. Ввести динамический массив целых чисел, количество элементов массива равно N. Перевернуть элементы массива, начиная с элемента с индексом P, заканчивая элементом с индексом Q. Составить блок-схему алгоритма и вписать код на C++

Динамический массив – структура данных, которая может изменять свой размер во время выполнения программы и освобождать ненужные ячейки памяти.

Объявляется с использованием указателя на первый элемент массива: `int* arr = new int[size];`

Оператор `new[]` помогает выделить память в динамической памяти: запрашивает соответствующее количество байтов у ОС и возвращает указатель в начало области.

`[]` - интерпретирует область памяти как последовательность элементов массива.

`delete[]` - используется, чтобы не произошла утечка памяти.

Обращение к элементу 0: `arr[0]`, `*arr`

Обращение к элементу 1: `*(arr + 1)`

```
#include <iostream>
```

```
// Функция для переворачивания элементов массива от P до Q
```

```
void reverse(int* array, int start, int end) {
```

```
    while (start < end) {
```

```
        int temp = array[start];
```

```
        array[start] = array[end];
```

```
        array[end] = temp;
```

```
        start++;
```

```
        end--;
```

```
    }
```

```
}
```

```
int main() {
```

```
    int N = 5; // Количество элементов
```

```
    int* mas = new int[N]; // Выделение памяти под динамический массив
```

```
    // Инициализация массива (для примера)
```

```
    for (int i = 0; i < N; i++) {
```

```
        mas[i] = i + 1;
```

```
    }
```

```
    int P = 1, Q = 3; // Пример индексов P и Q
```

```
    reverse(mas, P, Q); // Переворачивание элементов от P до Q (включительно)
```

```
    // Вывод элементов массива для проверки
```

```
    for (int i = 0; i < N; i++) {
```

```
        std::cout << mas[i] << " ";
```

```
    }
```

```
    delete[] mas; // Освобождение памяти массива
```

```
    return 0;  
}
```

2. Структуры: потоковый ввод-вывод, уровни ввода-вывода, содержащиеся в библиотеке Си. В какой библиотеке содержатся функции ввода-вывода? Создать структуру "Человек", которая содержит 4 поля. Привести блок кода определения структуры и функцию инициализации экземпляра данными с клавиатуры.

```
#include <stdio.h>
```

```
struct Person {  
    string name;  
    int age;  
    double weight;  
    double height;  
};  
  
void inputPerson(Person& p) {  
    cin >> p.name;  
    cin >> p.age;  
    cin >> p.weight;  
    cin >> p.height;  
}
```

Экзаменационный билет №16

1. Функции: Понятие функции, параметр функции, функция с переменным числом параметров, перегрузка функции. Объяснить термин “перегруженные функции”. Можно ли перегружать параметризованные функции?

Функция - это блок кода, который выполняет определенную задачу и может принимать параметры на вход.

Параметр функции - это значение, которое передается в функцию в качестве аргумента при ее вызове. Параметры могут быть обычными переменными, указателями или ссылками.

Функция с переменным числом параметров - это функция, которая может принимать любое количество параметров, например, функция printf().

Перегрузка функции - это создание двух или более функций с одинаковым именем, но разными параметрами.

Да, параметризованные функции могут быть перегружены. В C++ можно перегружать функции с параметрами разных типов и количеством, а также функции-шаблоны, работающие с разными типами параметров.

2. Стек: понятие стека, отличие от списка, добавление и удаление элементов в стек. Для чего используется стек. Дан стек и строка, состоящая из открывающих и закрывающих скобок ('(', ')', '[', ']', '{', '}') и пробелов. Написать блок кода, определяющий, является ли данная строка корректной скобочной последовательностью.

```
#include <iostream>
#include <stack>
#include <string>

bool VseNorm(const std::string& str) {
    std::stack<char> stck;
    for (char ch : str) { // Цикл, перебирающий все символы строки
        switch (ch) {
            case '[':
            case '{':
            case '(':
                stck.push(ch);
                break;
            case ']':
                if (stck.empty() || stck.top() != '[') {
                    return false;
                }
                stck.pop();
                break;
            case '}':
                if (stck.empty() || stck.top() != '{') {
                    return false;
                }
                stck.pop();
                break;
        }
    }
    return stck.empty();
}
```



```

        case ')':
            if (stck.empty() || stck.top() != '(') {
                return false;
            }
            stck.pop();
            break;
        case ' ':
            break;
        default:
            return false;
    }
}
return stck.empty();
}

```

```

int main()
{
    std::string str;
    std::cin >> str;
    if (VseNorm(str)) {
        std::cout << "Vse norm:");
    }
    else
    {
        std::cout << "Ne norm:(";
    }
    return 0;
}

```

Экзаменационный билет №17

1. Шаблонные функции: Понятие шаблонной функции, смысл использования шаблонных функций. Перечислите основные свойства параметров шаблона функций. Пример создания шаблонной функции. Написать на С++ шаблонные функции и основную программу, которая их вызывает:

- для массива целых чисел находит максимальный элемент;
- для строки находит длину самого большого слова.

Шаблонная функция – функция, которая позволяет работать с различными типами данных, не изменяя структуру. Смысл: избегание дублирования операций, гибкость в работе с разными типами данных, удобство в обобщении алгоритма. Свойства: тип параметра, значение по умолчанию, ограничения параметра.

```
template<typename T>
T findMax(T arr[], int size)
{
    T max = arr[0];
    for (int i = 1; i < size; i++)
        if (arr[i] > max)
            max = arr[i];

    return max;
}

template<>
std::string findMax(std::string arr[], int size)
{
    std::string max = arr[0];
    for (int i = 1; i < size; i++)
        if (arr[i].length() > max.length())
            max = arr[i];

    return max;
}
```

2. Строки: Какие слова встречаются в строке по одному разу. Написать блок-схему с вписанным кодом на С++.

```
int in(std::vector<std::string> words, std::string word) // Функция, которая
считает
кол-во вхождений слова
{
    int tmp = 0;
    for (std::string tmpWord : words) {
        if (word == tmpWord) tmp += 1; }
    return tmp;
}

int main()
{
    std::string str;
    std::getline(std::cin, str);
    str += ' ';
    std::vector<std::string> words;
    std::string word;
    for (int i = 0; i < str.length(); ++i) {
```

```
if (str[i] != ' ') word += str[i];  
else { words.push_back(word); word = ""; }  
}  
for (std::string tmpWord : words) {  
if (in(words, tmpWord) == 1) std::cout << tmpWord << ' ' ;  
  
}  
return 0;
```

```
}
```

Экзаменационный билет №18

1. Дружественные функции: Понятие дружественной функции. Являются ли дружественные функции, описанные в параметризованном классе, параметризованными? Пример создания дружественной функции. Написать на C++ дружественные функции и основную программу, которая их вызывает.

Дружественная функция - это функция, которая не является методом класса, но имеет доступ к приватным и защищенным членам этого класса, в том числе ко всем его экземплярам. Дружественные функции, описанные в параметризованном классе, не являются параметризованными, так как они имеют доступ к приватным и защищенным членам класса, независимо от его параметров шаблона.

```
class Point
{
private:
    double x;
public:
    Point(double _x) : x(_x) {}
    friend double distance(Point& p1, Point& p2);
};

double distance(Point& p1, Point& p2)
{
    double dx = p1.x - p2.x;
    return sqrt(dx * dx);
}

int main()
{
    Point p1(5);
    Point p2(6);
    cout << distance(p1, p2) << endl;
    return 0;
}
```

2. Структуры: потоковый ввод-вывод, уровни ввода-вывода, содержащиеся в библиотеке Си. В какой библиотеке содержатся функции ввода-вывода? Создать структуру "Покупатель", которая содержит 4 поля. Привести блок кода создания структуры и загрузки в двоичный файл.

В C++ потоковый ввод-вывод реализуется с помощью объектов класса `std::iostream`, который является частью стандартной библиотеки C++ (`stdlib.h`).

Уровни ввода-вывода в библиотеке C++

1. **Общий уровень** (`std::iostream`): Предоставляет методы для работы с потоками ввода-вывода.
2. **Высокоуровневые потки** (`std::cin`, `std::cout`)
3. **Специализированные потоки**: Например, `std::filebuf` для работы с файлами, `std::stringstream` для встроенных (в памяти) буферов ввода/вывода, `std::sstream` для строковых потоков (иногда называются строковыми буферами)

```
#include <iostream>
#include <fstream>
#include <string>
```

```
struct buyer {
```

```
    std::string name;
    std::string surname;
    std::string address;
    int age;
};

int main() {
    // Создание структуры
    buyer buyer1;
    buyer1.name = "Иван";
    buyer1.surname = "Иванов";
    buyer1.address = "ул. Ленина, д. 5";
    buyer1.age = 30;

    // Загрузка структуры в двоичный файл
    std::ofstream outFile("buyers.bin", std::ios::binary);
    return 0;
}
```

Экзаменационный билет №19

1. Строковые и символьные переменные в C++. Передача строк в функцию как фактических параметров. Привести определения терминов и пример кода на C++.

Передача строк в функцию как фактических параметров:

Строки могут быть переданы в функцию как фактические параметры двумя способами: по значению и по ссылке.

При передаче строки *по значению* создается копия строки, которая затем передается в функцию. Это может занимать много времени и памяти для больших строк.

```
void print(const string str) {...}
```

При передаче строки *по ссылке* передается адрес строки, а не копия, что позволяет избежать затрат на создание копий и работать с оригинальной строкой.

```
void print(const string& str) {...}
```

2. Напишите функцию, которая будет принимать на вход указатель на эту хеш-таблицу и строку S, а затем искать в таблице первый ключ, у которого соответствующее значение полностью совпадает со строкой S. Если такой ключ найден, функция должна вернуть его значение. Если такого ключа нет, функция должна вернуть -1. Например, если в таблице есть ключ 45 со значением "abcd" и ключ 57 со значением "ab", а на вход функции подана строка S="ab", то функция должна вернуть значение 57.

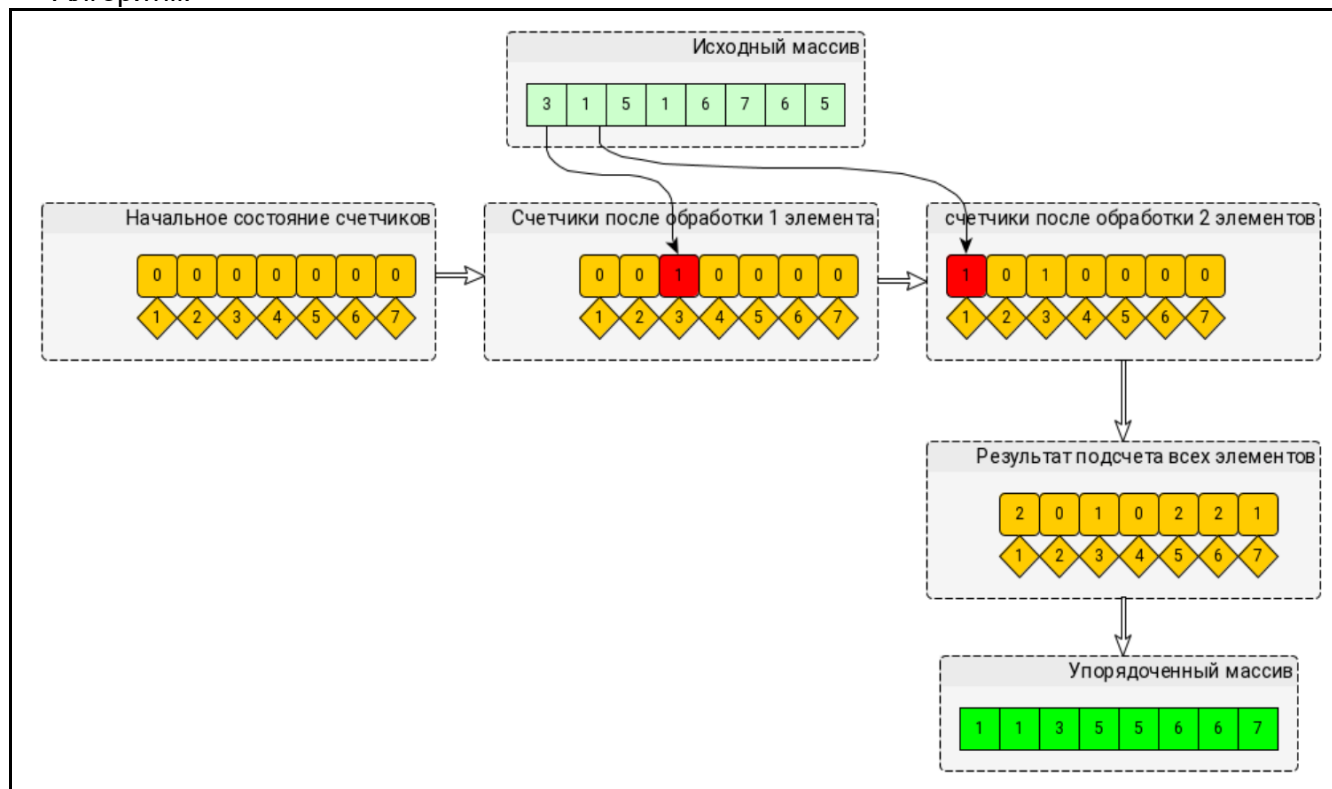
```
int findKey(hash_table* tab, string s)
{
    for (int i = 0; i < tab->size; i++)
        if (tab->strings[i] == s)
            return tab->GetKey(i);

    return -1;
}
```

Экзаменационный билет №20

1. Составьте блок-схему с вписанным кодом на C++: метод сортировки подсчетом. Ввести динамический массив целых чисел, количество элементов массива равно N. Отсортировать элементы массива методом «Подсчета».

Алгоритм:



На заданном массиве:

1)

8	7	6	5	16	5	10	11
---	---	---	---	----	---	----	----

 $\max = 16$

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

2)

0	0	0	0	1	1	1	0	1	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

5	6	7	9	10	11	15	16
---	---	---	---	----	----	----	----

Код:

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> A = { 3, 3, 3, 1, 1, 0, 2, 2 };

    int max = A[0]; // Находим максимальный элемент
    for (int i = 1; i < A.size(); i++) {
        if (A[i] > max) {
            max = A[i];
        }
    }
}
```

```

    }
}

// Создаем вектор, размер которого равен максимальному элементу + 1, т.к у
нас индексы
vector<int> B(max + 1, 0);
for (int i = 0; i < A.size(); i++) { // Идем по исходному массиву. Если
встретили 0, то к нулевому индексу массива B += 1, если встретили 1, то к 1-ому
индексу массива B += 1
    int index = A[i];
    B[index] += 1;
}

// Переформировываем массив
int k = 0;
for (int i = 0; i < max + 1; i++) {
    for (int j = 0; j < B[i]; j++) {
        A[k] = i;
        k++;
    }
}

// Выводим
for (int i = 0; i < A.size(); i++) {
    cout << A[i] << " ";
}

return 0;
}

```

2. Функция `getline`. Показать её использование в C++ на примере задания: вывести все слова из строки, которые содержат букву В.

```

#include <iostream>
#include <string>

int main() {
    std::string input;
    input += ' ';
    std::getline(std::cin, input);

    std::string word = "";

    for (char c : input) {
        if (c != ' ') {
            word += c; // Формируем текущее слово
        }
        else {

```



```
        for (char letter : word) {
            if (letter == 'B') {
                std::cout << word << std::endl; // Выводим слово, если
содержит букву B
                break;
            }
        }
        word = ""; // Очищаем текущее слово для следующей итерации
    }
}

return 0;
}
```

Экзаменационный билет №21

1. Интерполяционный метод поиска: В массиве найти заданный элемент равный z методом интерполяционного поиска. Реализовать фрагмент кода на C++ интерполяционным методом поиска: Дан отсортированный массив целых чисел и искомое число. Определить, есть ли искомое число в массиве. Пример: массив [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], искомое число 7. Ответ: есть.

```
bool InterpolationSearch(vector<int>& arr, int num) {
    int low = 0, high = arr.size()-1;
    int index;
    while (arr[low] < num && arr[high] > num) {
        if (arr[low] == arr[high])
            break;
        index = (num - arr[low]) * (low - high) / (arr[low] - arr[high]) +
low;
        if (arr[index] > num)
            high = index - 1;
        if (arr[index] < num)
            low = index + 1;
        else
            return true;
    }
    return false;
}
```

2. Структуры данных в C++: описать и сравнить принципы организации работы списков, стеков и очередей. Добавление и удаление элементов. Работа с указателями.

Списки представляют собой последовательность узлов, связанных между собой указателями. Они могут быть однонаправленными, когда каждый узел содержит указатель только на следующий узел, или двунаправленными, когда каждый узел содержит указатели на следующий и предыдущий узлы.

Добавление и удаление элементов в списке может осуществляться быстро, т.к. не требуется перемещение остальных элементов.

Стек представляет собой структуру данных, которая работает по принципу «последний вошел - первый вышел».

Это означает, что последний элемент, добавленный в стек, будет первым элементом, который будет удален из стека.

Очередь представляет собой структуру данных, которая работает по принципу "первый вошел - первый вышел".

Это означает, что первый элемент, добавленный в очередь, будет первым элементом, который будет удален из очереди.

Добавление и удаление элементов в структуры данных, такие как списки, стеки и очереди, обычно осуществляется с помощью указателей на элементы.

Экзаменационный билет №22

1. Функции в C++: привести пример функций с переменным числом параметров и перегруженных функций. Написать на C++ перегруженные функции и основную программу, которая их вызывает:
- для массива целых чисел находит максимальный элемент;
 - для строки находит длину самого большого слова.

```
int findMax(const int arr[], int size) {
    int maxElement = arr[0];
    for (int i = 1; i < size; ++i) {
        if (arr[i] > maxElement) {
            maxElement = arr[i];
        }
    }
    return maxElement;
}

int findMaxLength(const std::string& str) {
    std::string word;
    int maxLength = 0;
    std::string s(str);

    s += ' '; // Добавляем пробел в конце для обработки последнего слова

    for (char c : s) {
        if (c != ' ') {
            word += c;
        } else {
            maxLength = std::max(maxLength, word.length());
            word = "";
        }
    }

    return maxLength;
}
```

2. Строки: Какие слова встречаются в строке по одному разу. Написать блок-схему с вписанным кодом на C++.

```
#include <iostream>
#include <vector>
#include <string>

int in(std::vector<std::string> words, std::string word) // Функция,
которая считает
кол-во вхождений слова
{
    int tmp = 0;
```

```
        for (std::string tmpWord : words) {
            if (word == tmpWord) tmp += 1; }
        return tmp;
    }

int main()
{
    std::string str;
    std::getline(std::cin, str);
    str += ' ';
    std::vector<std::string> words;
    std::string word;
    for (int i = 0; i < str.length(); ++i) {
        if (str[i] != ' ') word += str[i];
        else { words.push_back(word); word = ""; }
    }
    for (std::string tmpWord : words) {
        if (in(words, tmpWord) == 1) std::cout << tmpWord << ' ';

    }
    return 0;
}
```

Экзаменационный билет №23

1. Метод поиска Кнута-Морриса-Пратта: В массиве найти заданный элемент равный z методом поиска Кнута-Морриса-Пратта. Реализовать фрагмент кода на C++ методом поиска Кнута-Морриса-Пратта: Найти все вхождения образца P в строку S. Пример: строка S "ababababababab", образец P "aba". Ответ: найдены вхождения в позициях 0, 2, 4, 6, 8, 10.

```
void KMP(string str, string word) {
    //str - искомое слово
    //создание массива для сдвига искомого слова
    vector<int>arr(str.size()); //кол-во элементов = размеру слова
    int j = 0, i = 1;
    //цикл заполняет массив пока не дойдет до конца слова
    while (i < str.size()) {
        if (str[i] != str[j]) {
            if (j == 0) {
                arr[i] = 0;
                i++;
            }
            else
                j = arr[j - 1];
        }
        else {
            arr[i] = j + 1;
            i++; j++;
        }
    }
    //word - строка, в которой ищется слово
    int m = str.size();
    int n = word.size();
    i = 0, j = 0;
    //поисковый цикл работает пока не конец слова
    while (i < n) {
        if (word[i] == str[j]) {
            i++; j++;
            if (j == m) {
                cout << "true";
                break;
            }
        }
        else {
            if (j > 0) j = arr[j - 1];
            else i++;
            if (i == n) cout << "false";
        }
    }
}
```

2. Написать рекурсивную функцию: получение последовательности чисел Фибоначчи, если количество элементов в последовательности равно N. Составить блок-схему алгоритма и вписать код на C++.

```
#include <iostream>

int fibonacci(int n) {
    if (n <= 1) {
        return n;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int N;
    std::cin >> N;

    for (int i = 0; i < N; i++) {
        std::cout << fibonacci(i) << " ";
    }

    return 0;
}
```

Экзаменационный билет №24

1. Метод поиска Бойера-Мура-Хорспула: В массиве найти заданный элемент равный z методом поиска Бойера-Мура-Хорспула. Реализовать фрагмент кода на C++ методом поиска Бойера-Мура-Хорспула: Найти все вхождения образца Р в строку S. Пример: строка S "ababababababab", образец Р "aba". Ответ: найдены вхождения в позициях 0, 2, 4, 6, 8, 10.

```
int BMSearch(string str, string substr)
{
    int sl, ssl;

    sl = str.size();
    ssl = substr.size();

    if (sl != 0 && ssl != 0)
    {
        int i, pos;
        int bias[256];

        for (i = 0; i < 256; i++)
            bias[i] = ssl;

        for (i = ssl - 2; i >= 0; i--)
            if (bias[int((char)substr[i])] == ssl)
                bias[int((char)substr[i])] = ssl - i - 1;

        pos = ssl - 1;

        while (pos < sl)
            if (substr[ssl - 1] != str[pos])
                pos += bias[int((char)str[pos])];
            else
                for (i = ssl - 1; i >= 0; i--)
                    if (substr[i] != str[pos - ssl + i + 1])
                    {
                        pos += bias[int((char)str[pos - ssl + i + 1])];
                        break;
                    }
                else
                    if (i == 0)
                        return pos - ssl + 1;
    }
    return -1;
}
```

2. Написать рекурсивную функцию: считывать введенные с клавиатуры числа до тех пор, пока не будет введен ноль. Составить блок-схему алгоритма и вписать код на C++.

```
void func() {
    int n;
    std::cin >> n; // Считываем число с клавиатуры

    if (n != 0) {
        func(); // Рекурсивный вызов функции, если число не равно нулю
    }
}
```

Экзаменационный билет №25

1. Написать на C++ функцию: чтение структур из двоичного файла с использованием блочного ввода-вывода и сохранение их в стек.

```
void readStructsToStack(string filename, stack<MyStruct>& myStack) {
    ifstream file;
    file.open(filename, ios::binary);

    streampos begin = file.tellg();
    file.seekg(0, ios::end);
    streampos end = file.tellg();
    int fileSize = end - begin;

    int sizeOfStruct = sizeof(MyStruct);
    int numOfStructs = fileSize / sizeOfStruct;

    file.seekg(0, ios::beg);

    for (int i = 0; i < numOfStructs; i++) {
        MyStruct myStruct;
        file.read(reinterpret_cast<char*>(&myStruct), sizeOfStruct);
        myStack.push(myStruct);
    }

    file.close();
}
```

```
#include <iostream>
#include <fstream>
#include <vector>

// Пример структуры, которая будет сохраняться в файле
struct Data {
    int id;
    double value;
};

void readStructsFromFile(const std::string& filename, std::stack<Data>&
stack) {
    std::ifstream file(filename, std::ios::binary | std::ios::in);

    if (file.is_open()) {
        while (!file.eof()) {
            Data data;
            file.read(reinterpret_cast<char*>(&data), sizeof(Data));
            stack.push(data);
        }
        file.close();
    } else {
        std::cerr << "Unable to open file";
    }
}
```



```

int main() {
    std::stack<Data> dataStack;
    readStructsFromFile("data.bin", dataStack);

    // Вывод данных из стека
    while (!dataStack.empty()) {
        Data data = dataStack.top();
        std::cout << "ID: " << data.id << ", Value: " << data.value <<
std::endl;
        dataStack.pop();
    }

    return 0;
}

```

2. Метод поиска Кнута-Морриса-Пратта: В массиве найти заданный элемент равный z методом поиска Кнута-Морриса-Пратта. Реализовать фрагмент кода на C++ методом поиска Кнута-Морриса-Пратта: Найти все вхождения образца P в строку S. Пример: строка S "ababababababab", образец P "aba". Ответ: найдены вхождения в позициях 0, 2, 4, 6, 8, 10.

```

void KMP(string str, string word) {
    //str - искомое слово
    //создание массива для сдвига искомого слова
    vector<int>arr(str.size()); //кол-во элементов = размеру слова
    int j = 0, i = 1;
    //цикл заполняет массив пока не дойдет до конца слова
    while (i < str.size()) {
        if (str[i] != str[j]) {
            if (j == 0) {
                arr[i] = 0;
                i++;
            }
            else
                j = arr[j - 1];
        }
        else {
            arr[i] = j + 1;
            i++; j++;
        }
    }
    //word - строка, в которой ищется слово
    int m = str.size();
    int n = word.size();
    i = 0, j = 0;
    //поисковый цикл работает пока не конец слова
    while (i < n) {
        if (word[i] == str[j]) {
            i++; j++;
        }
    }
}

```

```
        if (j == m) {  
            cout << "true";  
            break;  
        }  
    }  
    else {  
        if (j > 0) j = arr[j - 1];  
        else i++;  
        if (i == n) cout << "false";  
    }  
}  
}
```

Экзаменационный билет №26

1. Структуры: потоковый ввод-вывод, уровни ввода-вывода, содержащиеся в библиотеке Си. В какой библиотеке содержатся функции ввода-вывода? Создать структуру "Абитуриент", которая содержит 4 поля. Привести блок кода создания структуры и загрузки в двоичный файл.

Функции ввода-вывода содержатся в стандартной библиотеке, которая включает заголовочные файлы <stdio.h> для стандартного ввода-вывода.

```
#include <iostream>
#include <fstream>

// Определение структуры "Абитуриент"
struct Applicant {
    std::string name;
    int age;
    std::string city;
    double score;
};

int main() {
    // Создание экземпляра структуры "Абитуриент"
    Applicant student;
    student.name = "Иванов";
    student.age = 18;
    student.city = "Москва";
    student.score = 87.5;

    // Открытие двоичного файла для записи
    std::ofstream outFile("applicants.dat", std::ios::binary);
    if (!outFile) {
        std::cerr << "Ошибка открытия файла!" << std::endl;
        return 1;
    }

    // Запись структуры в файл
    outFile.write(reinterpret_cast<const char*>(&student), sizeof(Applicant));
    outFile.close();

    std::cout << "Данные успешно записаны в файл в двоичном формате." <<
std::endl;

    return 0;
}
```

2. Написать рекурсивную функцию: получение последовательности чисел Фибоначчи, если количество элементов в последовательности равно N. Составить блок-схему алгоритма и вписать код на C++.

```
#include <iostream>

int fibonacci(int n) {
    if (n <= 1) {
        return n;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int N;
    std::cin >> N;

    for (int i = 0; i < N; i++) {
        std::cout << fibonacci(i) << " ";
    }

    return 0;
}
```

Экзаменационный билет №27

1. Структуры: поток в теме блокового ввода-вывода, режимы для открытия поток. В какой библиотеке содержатся функции ввода-вывода? Создать структуру "Студент", которая содержит 4 поля: ФИО, домашний адрес, группа, оценки. Написать на C++ функцию, считывающую из файла структуры и выводящую в консоль данные о студентах со средним баллом по оценкам, большим 4,5.

```
void readStudentsFromFile(string filename) {
    ifstream file(filename, ios::in | ios::binary);
    Student s;
    int count = 0;
    while (file.read((char*)&s, sizeof(Student))) {
        float avg = 0;
        for (int i = 0; i < 5; i++) {
            avg += s.marks[i];
        }
        avg /= 5.0f;
        if (avg > 4.5f) {
            count++;
            cout << "Student #" << count << endl;
            cout << "FIO: " << s.fio << endl;
            cout << "Address: " << s.address << endl;
            cout << "Group: " << s.group << endl;
            cout << "Average mark: " << avg << endl << endl;
        }
    }
    file.close();
}
```

2. Написать рекурсивную функцию: «Ханойская башня» для трёх стержней на C++. Составить блок-схему алгоритма и вписать код на C++.

```
#include <iostream>

void towerOfHanoi(int n, int start, int finish, int temp) {
    if (n == 1) {
        std::cout << start << " to " << finish << std::endl;
        return;
    }
    towerOfHanoi(n - 1, start, temp, finish);
    std::cout << start << " to " << finish << std::endl;
    towerOfHanoi(n - 1, temp, finish, start);
}

int main() {
    towerOfHanoi(3, 1, 3, 2); // Вызов функции Ханойской башни

    return 0;
}
```

Экзаменационный билет №28

1. Списки: однонаправленный список, отличие от массива, добавление элементов в список. Куда быстрее добавляется элемент - в список или массив. Напишите функцию, которая удаляет из уже имеющегося односвязного списка повторяющиеся элементы.

Однонаправленный список - это структура данных, которая состоит из узлов, каждый из которых содержит данные и ссылку на следующий узел. Добавление происходит за счет изменения указателя на узел.

```
void removeDuplicates(Node* head) {  
    Node* current = head;  
  
    while (current != nullptr) {  
        Node* runner = current;  
  
        while (runner->next != nullptr) {  
            if (runner->next->data == current->data) {  
                Node* duplicate = runner->next;  
                runner->next = runner->next->next;  
                delete duplicate;  
            } else {  
                runner = runner->next;  
            }  
        }  
  
        current = current->next;  
    }  
}
```

2. Хеш-таблицы: понятие хеш-таблицы, поиск элементов в хеш-таблице, метод цепочек и что он решает.

Хеш-таблица – абстрактная структура данных с функционалом массивов и списков.

- Неупорядоченная структура
- Состоит из хеш-функции (хеш-кода) и массива для хранения
- Хеш-функция возвращает число, позволяющее определить место для размещения элемента.

Поиск элемента в хеш-таблице осуществляется путем преобразования ключа элемента с помощью хеш-

функции в индекс массива, где он должен быть расположен. Затем выполняется поиск элемента по этому

индексу. Если в этой ячейке находится более одного элемента (конфликт), используется метод разрешения

коллизий (например, метод цепочек).

Коллизия – разные данные, прошедшие одну и ту же функцию, с одинаковым числовым значением.

- Метод открытой адресации (увеличить хэш на 1 и попробовать новую ячейку)
- Метод цепочек (хеш-таблица становится массивом списка)

Экзаменационный билет №29

1. Списки: двунаправленный список, отличие от массива, добавление элементов в список. Куда быстрее добавляется элемент - в список или массив? Напишите функцию, которая удаляет из уже имеющегося двунаправленного списка повторяющиеся элементы.

Двунаправленный список - это последовательность, где каждый элемент ссылается на предыдущий и следующий элементы. В отличие от массива, список - это динамическая последовательность элементов, элементы могут быть распределены по разным блокам памяти.

Добавление элемента в список выполняется быстрее чем в массив, так как не требует перемещения остальных элементов. Удаление также происходит в константное время, потому что ссылки на элементы обновляются на лету.

```
void removeDuplicates(Node* head) {
    Node* current = head;

    while (current != nullptr) {
        Node* runner = current->next;

        while (runner != nullptr) {
            Node* nextNode = runner->next;
            if (current->data == runner->data) {
                if (runner == head) {
                    head = runner->next;
                }
                if (runner->next != nullptr) {
                    runner->next->prev = runner->prev;
                }
                if (runner->prev != nullptr) {
                    runner->prev->next = runner->next;
                }
                delete runner;
            }
            runner = nextNode;
        }

        current = current->next;
    }
}
```

2. Составьте блок-схему с вписанным кодом на C++: метод сортировки подсчетом. Ввести динамический массив целых чисел, количество элементов массива равно N. Отсортировать элементы массива методом «Подсчета».

Алгоритм:


```

индексу массива B += 1
    int index = A[i];
    B[index] += 1;
}

// Переформировываем массив
int k = 0;
for (int i = 0; i < max + 1; i++) {
    for (int j = 0; j < B[i]; j++) {
        A[k] = i;
        k++;
    }
}

// Выводим
for (int i = 0; i < A.size(); i++) {
    cout << A[i] << " ";
}

return 0;
}

```

Экзаменационный билет №30

1. Стек: понятие стека, отличие от списка, добавление и удаление элементов в стек. Для чего используется стек. Привести пример организации работы на C++.

Стек - это абстрактная структура данных, которая работает по принципу «последним пришел - первым вышел».

Работая со стеком, можно добавлять элементы на вершину стека и удалять их только с вершины.

Отличие стека от списка заключается в том, что *в стеке доступ осуществляется только к вершине стека*, в то время как *в списке можно получить доступ к элементам в любом месте*.

Стек используется для управления вызовами функций, в рекурсиях, управление памятью.

```
#include <iostream>

// Структура для узла стека
struct Node {
    int data; // Данные узла стека
    Node* next; // Указатель на следующий узел
};

Node* top = nullptr; // Указатель на вершину (и одновременно дно) стека (создаем стек)
int size = 0; // Размер стека

// Функция для вставки нового элемента на заданную позицию в стеке
void insert(int value, int position) {
    Node* temp = new Node; // Создаем новый узел
    temp->data = value; // Устанавливаем значение в новый узел

    if (position == 0) { // Вставка в начало стека
        temp->next = top; // Связываем новый узел со стеком
        top = temp; // Обновляем вершину стека
    }
    else {
        Node* current = top; // Определяем текущий узел как вершину стека
        // Поиск узла перед позицией вставки
        for (int i = 1; i < position; i++) {
            current = current->next; // Перемещаемся к узлу перед позицией вставки
        }
        temp->next = current->next; // Связываем новый узел с последующим узлом
        current->next = temp; // Обновляем порядок узлов в стеке
    }
}
```

```

    size++; // Увеличиваем размер стека после вставки
}

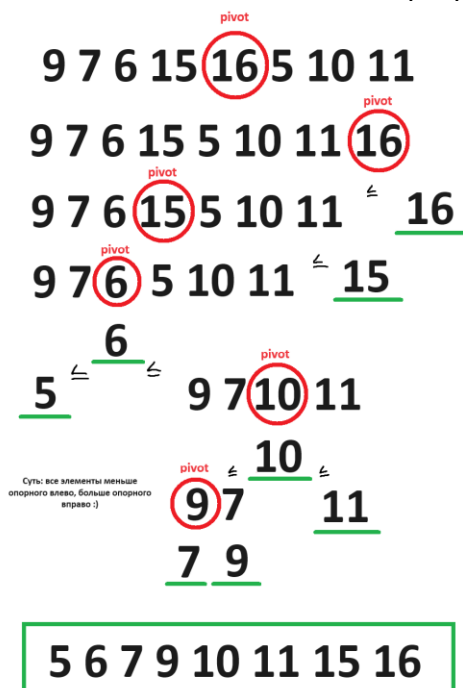
int main() {
    // Добавляем элементы в стек с использованием функции вставки на заданную
    позицию
    insert(5, 0); // Вставка элемента 5 на позицию 0
    insert(10, 1); // Вставка элемента 10 на позицию 1

    // Вывод элементов стека
    Node* current = top;
    if (current == nullptr) {
        std::cout << "Стек пуст" << std::endl;
    } else {
        std::cout << "Элементы стека:" << std::endl;
        while (current != nullptr) {
            std::cout << current->data << std::endl;
            current = current->next;
        }
    }

    return 0;
}

```

2. Составьте блок-схему с вписанным кодом на C++: метод сортировки Хоара. Ввести динамический массив целых чисел, количество элементов массива равно N. Отсортировать элементы массива методом «Хоара». Массив содержит следующие элементы: 9 7 6 15 16 5 10 11. Показать на заданных элементах последовательность шагов сортировки.



```
int partOfSortHoara(int start, int end, vector<int>& vect) {
    int mid_element = vect[(start + end) / 2];
    int st = start, en = end;
    while (st <= en) {
        while (vect[st] < mid_element)
            st++;
        while (vect[en] > mid_element)
            en--;
        if (st <= en) {
            swap(vect[st], vect[en]);
            st++;
            en--;
        }
    }
    return st;
}

void sortHoara(int start, int end, vector<int>& vect) {
    if (end - start < 1) return;
    int border = partOfSortHoara(start, end, vect);
    sortHoara(start, border - 1, vect);
    sortHoara(border, end, vect);
}
```

Экзаменационный билет №31

1. Очередь: понятие очереди, отличие от списка, добавление и удаление элементов в очередь. Для чего используется очередь. Привести пример организации работы на C++.

Контейнер, который работает по принципу первый вошел — первым вышел. Первым всегда извлекается первый добавленный элемент.

Отличие от списка: добавление элементов – в конец, удаление элементов – из начала.

Очереди используются в обслуживании клиентов, в сети (передача кадров данных), в многозадачности.

```
#include <iostream>
#include <queue>
#include <string>
```

```
int main()
{
    std::queue<std::string> q;
    for (int i = 0; i < 3; i++)
    {
        std::string tmp;
        std::cin >> tmp;
        q.push(tmp);
    }
    std::cout << std::endl << "Внимание! Это очередь:" << std::endl;
    while (!q.empty())
    {
        std::cout << q.front() << std::endl; // Выводим первый элемент
        q.pop(); // Удаляем первый элемент из очереди
    }
    return 0;
}
```

2. Написать рекурсивную функцию: «Ханойская башня» для трёх стержней на C++. Составить блок-схему алгоритма и вписать код на C++.

```
#include <iostream>

void towerOfHanoi(int n, int start, int finish, int temp) {
    if (n == 1) {
        std::cout << start << " to " << finish << std::endl;
        return;
    }
    towerOfHanoi(n - 1, start, temp, finish);
    std::cout << start << " to " << finish << std::endl;
    towerOfHanoi(n - 1, temp, finish, start);
}
```

```
int main() {  
    towerOfHanoi(3, 1, 3, 2); // Вызов функции Ханойской башни  
  
    return 0;  
}
```

Экзаменационный билет №32

1. Дан стек и строка, состоящая из открывающих и закрывающих скобок ('(', ')', '[', ']', '{', '}') и пробелов. Написать блок кода, определяющий, является ли данная строка корректной скобочной последовательностью.

```
#include <iostream>
#include <stack>
#include <string>

bool VseNorm(const std::string& str) {
    std::stack<char> stck;
    for (char ch : str) {
        switch (ch) {
            case '[':
            case '{':
            case '(':
                stck.push(ch);
                break;
            case ']':
                if (stck.empty() || stck.top() != '[') {
                    return false;
                }
                stck.pop();
                break;
            case '}':
                if (stck.empty() || stck.top() != '{') {
                    return false;
                }
                stck.pop();
                break;
            case ')':
                if (stck.empty() || stck.top() != '(') {
                    return false;
                }
                stck.pop();
                break;
            case ' ':
                break;
            default:
                return false;
        }
    }
    return stck.empty();
}

int main() {
    std::string str;
    std::cout << "Введите строку: ";
    std::cin >> str;
    if (VseNorm(str)) {
```

```

        std::cout << "Vse norm:);";
    } else {
        std::cout << "Ne norm:(";
    }
    return 0;
}

```

2. Строки: Какие слова встречаются в строке по одному разу. Написать блок-схему с вписанным кодом на C++.

```

#include <iostream>
#include <vector>
#include <string>

int in(std::vector<std::string> words, std::string word) // Функция, которая
считает
кол-во вхождений слова
{
    int tmp = 0;
    for (std::string tmpWord : words) {
        if (word == tmpWord) tmp += 1; }
    return tmp;
}

int main()
{
    std::string str;
    std::getline(std::cin, str);
    str += ' ';
    std::vector<std::string> words;
    std::string word;
    for (int i = 0; i < str.length(); ++i) {
        if (str[i] != ' ') word += str[i];
        else { words.push_back(word); word = ""; }
    }
    for (std::string tmpWord : words) {
        if (in(words, tmpWord) == 1) std::cout << tmpWord << ' ';

    }
    return 0;
}

```


Экзаменационный билет №33

Хеш-таблицы: понятие хеш-таблицы, поиск элементов в хеш-таблице, коллизия и какими методами ее можно разрешить.

Хеш-таблица – абстрактная структура данных с функционалом массивов и списков.

- Неупорядоченная структура
- Состоит из хеш-функции (хеш-кода) и массива для хранения
- Хеш-функция возвращает число, позволяющее определить место для размещения элемента.

Поиск элемента в хеш-таблице осуществляется путем преобразования ключа элемента с помощью хеш-

функции в индекс массива, где он должен быть расположен. Затем выполняется поиск элемента по этому

индексу. Если в этой ячейке находится более одного элемента (конфликт), используется метод разрешения

коллизий (например, метод цепочек).

Коллизия – разные данные, прошедшие одну и ту же функцию, с одинаковым числовым значением.

- Метод открытой адресации (увеличить хэш на 1 и попробовать новую ячейку)
- Метод цепочек (хеш-таблица становится массивом списка)

1. Написать рекурсивную функцию: получение последовательности чисел Фибоначчи, если количество элементов в последовательности равно N. Составить блок-схему алгоритма и вписать код на C++.

```
#include <iostream>

int fibonacci(int n) {
    if (n <= 1) {
        return n;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int N;
    std::cin >> N;

    for (int i = 0; i < N; i++) {
        std::cout << fibonacci(i) << " ";
    }

    return 0;
}
```